

List in Python

Lists are used to store multiple elements in a single variable.

Lists is one of data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.

A list may contain Data Types like Integers, Strings, as well as Objects. Lists are mutable, and hence, they can be altered even after their creation.

How to create List:

Lists in Python can be created by just placing the sequence inside the square brackets[]. Each element of list should be separated by a comma(,).

Examples

```
lst = []  
  
print(lst)
```

List of numbers

```
lst = [10, 20, 30, 40]  
  
print(lst)
```

List of strings

```
lst = ["python", "django", "C++", "java"]  
  
print(lst)
```

Accessing list elements

```
lst = ["python", "django", "C++", "java"]  
  
print(lst[0])  
  
print(lst[2])
```

Repeated elements List

```
lst = [1, 2, 4, 4, 3, 3, 3, 6, 5]
```

```
print(lst)
```

Mixed List

```
lst = [1, 2, 'python', 4, 'django', 6, 'java']
```

```
print(lst)
```

Nested List

```
lst = [1, 2, 3, 4, [10, 20, 30, 40]]
```

```
print(lst)
```

```
print(lst[1])
```

```
print(lst[4])
```

```
print(lst[4][2])
```

```
print(lst[0:3])
```

List Reverse

```
lst = [10, 20, 30, 40, 50, 60, 70]
```

```
print(lst[-1::-1])
```

```
lst = [1, 2, 3, 4, 5, 'java', 'Python']
```

```
lst.reverse()
```

```
print(lst)
```

Finding Length of List

```
lst = [10, 20, 30, 40, 50, 60, 70]
```

```
print(len(lst))
```

List Iteration

```
lst = [10, 20, 30, 40, 50, 60, 70]
```

```
t = len(lst)
```

```
for x in range(t):
```

```
    print(lst[x])
```

```
lst = [10, 20, 30, 40, 50, 60, 70]
```

```
for x in lst:
```

```
    print(x)
```

Program to sum all the numbers in a list

```
lst = [10, 20, 30, 40, 50, 60, 70]
```

```
s = 0
```

```
for x in lst:
```

```
    s += x
```

```
print(s)
```

Program to find max number in a list

```
lst = [10, 20, 30, 40, 50, 60, 70]
```

```
m = lst[0]
```

```
for x in lst:
```

```
    if x > m:
```

```
        m = x
```

```
print(m)
```

Dynamic List

```
x = input("Enter any string: ")
```

```
lst = x.split()
```

```
print(lst)
```

***** Split splits a string into a list where each word is a list item:***

Adding Elements to a Python List

Elements can be added to the List by using the built-in `append()` function. Only one element at a time can be added to the list by using the `append()` method, for the addition of multiple elements with the `append()` method, loops are used.

Using Append

```
lst = []
```

```
print(lst)
```

```
# Adding Elements
```

```
lst.append(1)
```

```
lst.append(2)
```

```
lst.append(3)
```

```
print(lst)
```

```
# Adding elements using loop
```

```
for i in range(4, 8):
```

```
    lst.append(i)
```

```
print(lst)
```

```
# Addition of list to a list
```

```
lst2 = ['python', 'sunday']
```

```
lst.append(lst2)
```

```
print(lst)
```

Using insert() method

append() method only adds elements at the end of the List, for the addition of elements at the desired position, insert() is used. Unlike append() which takes only one argument, the insert() method requires two arguments

(position, value).

Example

```
lst = [1,2,3,4]
```

```
print(lst)
```

```
lst.insert(2, 10)
```

```
lst.insert(0, 'python')
```

```
print(lst)
```

Using extend()

This method is used to add multiple elements at the same time at the end of the list.

```
lst = [1, 2, 3, 4]
```

```
print(lst)
```

```
lst.extend([5, 'python', 'java'])
```

```
print(lst)
```

Removing Elements from the List

Using remove()

Elements can be removed from the List by using the built-in remove() function but an Error arises if the element doesn't exist in the list.

Remove() method only removes one element at a time, to remove a range of elements, the iterator is used. The remove() method removes the specified item.

Example

```
lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
print(lst)
```

```
# Removing elements from List
```

```
lst.remove(2)
```

```
lst.remove(3)
```

```
print(lst)
```

Using pop()

pop() function is used to remove and return an element from the list, but by default it removes only the last element of the list, to remove an element from a specific position of the List, the index of the element is passed as an argument to the pop() method.

```
lst = [1, 2, 3, 4, 5]
```

```
# Removing element from the last
```

```
lst.pop()
```

```
print(lst)
```

```
# Removing element at a specific place
```

```
lst.pop(2)
```

```
print(lst)
```

List Methods

Function	Description
----------	-------------

Append()	Add an element to the end of the list
Extend()	Add all elements of a list to another list
Insert()	Insert an item at the defined index
Remove()	Removes an item from the list
Clear()	Removes all items from the list
Index()	Returns the index of the first matched item
Count()	Returns the count of the number of items passed as an argument
Sort()	Sort items in a list in ascending order
Reverse()	Reverse the order of items in the list
copy()	Returns a copy of the list
pop()	Removes and returns the item at the specified index. If no index is provided, it removes and returns the last item.

```
a= ["cat", "dog", "tiger"]
```

```
print(a.index("dog"))
```

```
fruits = ["Apple", "Mango", "Banana", "Cherry" , "Papaya", "Apple"]
```

```
print(fruits.count("Apple"))
```

```
lis = [2, 1, 3, 5, 3, 8]
```

```
lis.sort()
```

```
for x in lis:
```

```
    print(x)
```

```
lis = [2, 1, 3, 5, 3, 8]
```

```
lis2 = lis.copy()
```

```
print(lis2)
```

Built-in functions with List

Function	Description
-----------------	--------------------

reduce()	apply a particular function passed in its argument to all of the list elements stores the intermediate result and only returns the final summation value
----------	--

sum()	Sums up the numbers in the list
-------	---------------------------------

ord()	Returns an integer representing the Unicode code point of the given Unicode character
-------	---

cmp()	This function returns 1 if the first list is “greater” than the second list
-------	---

max()	return maximum element of a given list
-------	--

`min()` return minimum element of a given list

`all()` Returns true if all element is true or if the list is empty

`any()` return true if any element of the list is true. if the list is empty, return false

`len()` Returns length of the list or size of the list

`enumerate()` Returns enumerate object of the list

`accumulate()` apply a particular function passed in its argument to all of the list elements returns a list containing the intermediate results

`filter()` tests if each element of a list is true or not

Syntax : `sum(iterable, start)`

iterable : iterable can be anything list , tuples or dictionaries , but most importantly it should be numbers.

start : this start is added to the sum of numbers in the iterable. If start is not given in the syntax , it is assumed to be 0.

```
numbers = [1,2,3,4,5,1,4,5]
```

```
Sum = sum(numbers)
```

```
print(Sum)
```

```
Sum = sum(numbers, 10)
```

```
print(Sum)
```

Dictionary

What is a Dictionary in Python?

Dictionaries in Python is a data structure, used to store values in key:value format. This makes it different from lists, tuples, and arrays as in a dictionary each key has an associated value.

How to Create a Dictionary

In Python, a dictionary can be created by placing a sequence of elements within curly {} braces, separated by a 'comma'.

The dictionary holds pairs of values, one being the Key and the other corresponding pair element being its Key:value.

Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be immutable.

Dictionary keys are case sensitive, the same name but different cases of Key will be treated distinctly.

Example

```
d = {1: 'python', 2: 'java', 3: 'C++'}  
print(d)
```

Accessing Elements of a Dictionary

To access the items of a dictionary refer to its key name. Key can be used inside square brackets.

```
d = {1: "python", 2: "java", 3: "C++", "name": "Program"}  
print(d['name'])  
print(d[1])
```

Nested Dictionaries

Example

The code defines a nested dictionary named 'd' with multiple levels of key-value pairs. It includes a top-level dictionary with keys 1, 2, and 3. The value associated with key 3 is another dictionary with keys 'A,' 'B,' and 'C.'

```
d = {1: 'Program', 2: 'For', 3: {'A': 'Dictionary', 'B': 'In', 'C': 'Python'}}  
print(d)
```

Accessing an Element of a Nested Dictionary

```
d = {1: 'Program', 2: 'For', 3: {'A': 'Dictionary', 'B': 'In', 'C': 'Python'}}  
print(d[1])  
print(d[3]['A'])  
print(d[3]['C'])
```

Adding Elements to a Dictionary

One value at a time can be added to a Dictionary by defining value along with the key e.g.

```
Dict[Key] = 'Value'
```

Eg.

```
Dict = {}
```

```
print(Dict)
```

```
Dict[0] = 'Python'
```

```
Dict[2] = 'Java'
```

```
Dict[3] = 1
```

```
print(Dict)
```

Deleting Elements using 'del' Keyword

The items of the dictionary can be deleted by using the del keyword as given below.

```
d = {1: 'python', 2: 'java', 3: 'C++'}
```

```
print(d)
```

```
del(d[3])
```

```
print(d)
```

Dictionary Methods

Method	Description
--------	-------------

dict.clear()	Remove all the elements from the dictionary
--------------	---

dict.copy()	Returns a copy of the dictionary
-------------	----------------------------------

dict.items()	Returns a list containing a tuple for each key value pair
--------------	---

dict.keys()	Returns a list containing dictionary's keys
-------------	---

dict.update(dict2)	Updates dictionary with specified key-value pairs
--------------------	---

dict.values()	Returns a list of all the values of dictionary
---------------	--

pop(key)	Remove the element with specified key
----------	---------------------------------------

popitem()	Removes the last inserted key-value pair
-----------	--

Example

```
dict1 = {1: "Python", 2: "Java", 3: "C++", 4: "Dart"}  
  
dict2 = dict1.copy()  
  
print(dict2)  
  
dict1.clear()  
  
print(dict1)  
  
print(dict2.get(1))  
  
print(dict2.items())  
  
print(dict2.keys())  
  
dict2.pop(4)  
  
print(dict2)  
  
dict2.popitem()  
  
print(dict2)  
  
dict2.update({3: "Scala"})  
  
print(dict2)  
  
print(dict2.values())
```

Merge 2 dictionaries

```
dict1 = {'a': 10, 'b': 8}  
  
dict2 = {'d': 6, 'c': 4}  
  
dict2.update(dict1)
```

```
print(dict2)
```

List Data Type to Implement a Stack and Queue

Stack

A Stack is a linear data structure.

Stack stores elements in LIFO (Last In First Out) or FILO (First In Last Out). LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.

top

5
4
3
2
1

Bottom

Stack Operations

- 1) Push --- inserting an element
- 2) Pop ---- deleting an element
- 3) Peek ---- display the last element
- 4) Display ---- display stack

Example

```
l = []
```

```
while True:
```

```
    c = int(input("
```

```
        1. Insert element
```

```
        2. Pop element
```

```
        3. Peek element
```

```
        4. Display stack
```

```
        5. Exit
```

```
    "))
```

```
    if c == 1:
```

```
        n = input("Enter value: ")
```

```
        l.append(n)
```

```
        print(l)
```

```
    elif c == 2:
```

```
        if len(l) == 0:
```

```
            print("empty stack")
```

```
        else:
```

```
            p = l.pop()
```

```
            print(p)
```

```
            print(l)
```

```
elif c == 3:

    if len(l) == 0:

        print("empty stack")

    else:

        print("Last stack value: ", l[-1])

elif c == 4:

    print(l)

elif c == 5:

    break

else:

    print("Invalid input!!!")
```

Queue

A Queue is a linear data structure.

Queue stores elements in First in First Out (FIFO)

Back

Front

10	20	30	40	50		
----	----	----	----	----	--	--

Queue Operations

- 1) Enqueue --- inserting an element in queue
- 2) Dequeue ---- deleting an element from queue
- 3) Front ---- display the last element
- 4) Rear ---- display stack

Example

```
l = []
```

```
while True:
```

```
    c = int(input("
```

1. Enqueue element
2. Dequeue element
3. Front element
4. Last element

5. Display Queue

6. Exit

"))

if c == 1:

 n = input("Enter value: ")

 l.append(n)

 print(l)

elif c == 2:

 if len(l) == 0:

 print("empty queue")

 else:

 del l[0]

 print(l)

elif c == 3:

 if len(l) == 0:

 print("empty Queue")

 else:

 print("Frst Queue element: ", l[0])

elif c == 4:

 if len(l) == 0:

```
        print("empty Queue")

    else:

        print("Frst Queue element: ", l[-1])

elif c == 5:

    print(l)

elif c == 6:

    break

else:

    print("Invalid input!!!")
```

Tuple

Tuples are ordered Data Type

Tuples are used to store multiple items in a single variable.

Tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Set, and Dictionary, all with different qualities and usage.

A tuple is a collection which is ordered and unchangeable.

Tuples are written with round brackets.

Example

```
t = (10, 20, 30, 40)
```

Properties of Tuple Items

Tuple items are ordered, unchangeable, and allow duplicate values.

Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates

Since tuples are indexed, they can have items with the same value:

Iterate Tuple

```
t = (10, 20, 30, 40)
```

```
l = len(t)
```

```
for x in range(l):
```

```
    print(t[x])
```

```
t = ("abc", 34, True, 40, "hello")
```

```
for x in t:
```

```
    print(x)
```

Different Operations Related to Tuples

Concatenation

Nesting

Repetition

Slicing

Deleting

Finding the length

Multiple Data Types with tuples

Conversion of lists to tuples

Tuples in a Loop

Concatenation of Python Tuples

To Concatenation of Python Tuples, we will use plus operators(+).

```
t1 = (0, 1, 2, 3)
```

```
t2 = ('a', 'b')
```

```
print(t1 + t2)
```

Nesting of Python Tuples

A nested tuple in Python means a tuple inside another tuple.

```
t1 = (0, 1, 2, 3)
```

```
t2 = ('a', 'b')
```

```
t3 = (t1, t2)
```

```
print(t3)
```

Repetition in Python Tuples

We can create a tuple of multiple same elements from a single element in that tuple.

```
t = ('xyz',)*3
```

```
print(t)
```

Slicing Tuples in Python

Slicing a Python tuple means dividing a tuple into small tuples using the indexing method.

```
t = (0 ,1, 2, 3)
```

```
print(t[1:])
```

```
print(t[::-1])
```

```
print(t[2:4])
```

Deleting a Tuple in Python

Tuple can be deleted using 'del' keyword. The output will be in the form of error because after deleting the tuple, it will give a NameError.

* Remove individual tuple elements is not possible, but we can delete the whole Tuple using Del keyword.

```
t = ( 0, 1)
```

```
del t
```

```
print(t)
```

Finding the Length of a Tuple

To find the length of a tuple, we can use Python's len() function and pass the tuple as the parameter.

```
t = (10, 20)
```

```
print(len(t))
```

Converting a List to a Tuple

We can convert a list in Python to a tuple by using the tuple() constructor and passing the list as its parameters.

```
l = [10, 11, 12]
```

```
print(tuple(l))
```

Tuple Methods

`min()`

`max()`

`count()`

`index()`

`sum()`

Example

```
t = (10, 20, 30, 40, 50)
```

```
print(min(t))
```

```
print(max(t))
```

```
print(t.count(20))
```

```
print(t.index(40))
```

```
print(sum(t))
```

Set

Sets are used to store multiple items in a single variable.

Set is one of 4 built-in data types in Python used to store collections of data

A set is a collection which is

unordered,

unchangeable, (*items are unchangeable, but you can remove items and add new items.*)

and unindexed.

Sets are written with curly brackets.

Example

```
s = {"apple", "banana", "cherry"}
```

```
print(s)
```

Sets only have values and no key, so

Duplicates Not Allowed

Sets cannot have two items with the same value.

True and 1 is considered the same value:

False and 0 is considered the same value:

```
s = {"apple", "banana", "cherry", True, 1, 2, False, 0}
```

```
print(s)
```

Methods for Sets

1) Adding elements to Python Sets

Insertion in the set is done through the `set.add()` function,

```
s = {"apple", "banana", "cherry"}  
  
s.add("mango")  
  
print(s)
```

2) Union operation on Python Sets

Two sets can be merged using `union()` function or `|` operator

```
s1 = {"apple", "banana", "cherry"}  
  
s2 = {"abc", 10, 20}  
  
s3 = s1.union(s2)  
  
print(s)  
  
s4 = s1 | s2  
  
print(s4)
```

3) Intersection operation on Python Sets

This can be done through `intersection()` or `&` operator

```
s1 = set()
```

```
s2 = set()
```

```
for i in range(5):
```

```
    s1.add(i)
```

```
for i in range(3,9):
```

```
    s2.add(i)
```

```
s3 = s1.intersection(s2)
```

```
print(s3)
```

```
s4 = s1 & s2
```

```
print(s4)
```

4) Finding Differences of Sets in Python

To find differences between sets is done through difference() or – operator.

```
s1 = set()
```

```
s2 = set()
```

```
for i in range(5):
```

```
    s1.add(i)
```

```
for i in range(3,9):  
    s2.add(i)  
  
s3 = s1.difference(s2)  
  
print(s3)  
  
s4 = s1 - s2  
  
print(s4)
```

5) Clearing Python Sets

Set Clear() method empties the whole set inplace.

```
s1 = {1,2,3,4,5,6}  
  
print(s1)  
  
s1.clear()  
  
print(s1)
```

Operators for Sets

Sets and frozen sets support the following operators:

Operators	Description
-----------	-------------

<code>key in s</code>	containment check
<code>key not in s</code>	non-containment check
<code>s1 == s2</code>	s1 is equivalent to s2
<code>s1 != s2</code>	s1 is not equivalent to s2
<code>s1 <= s2</code>	s1 is subset of s2
<code>s1 < s2</code>	s1 is proper subset of s2
<code>s1 >= s2</code>	s1 is superset of s2
<code>s1 > s2</code>	s1 is proper superset of s2
<code>s1 s2</code>	the union of s1 and s2
<code>s1 & s2</code>	the intersection of s1 and s2
<code>s1 - s2</code>	the set of elements in s1 but not s2
<code>s1 ^ s2</code>	the set of elements in precisely one of s1 or s2